Rapport de Projet : League of Stones

Phase 3 (Authentification) et Phase 4 (Matchmaking)

Hyacinthe — Responsable Mobile

22 Mars 2026

1 Introduction

Ce document récapitule les travaux effectués durant la première semaine du projet *League of Stones*. En tant que responsable mobile et support sur l'authentification, j'ai implémenté la logique d'inscription et l'intégralité du système de matchmaking (Lobby) en respectant l'architecture Next.js (Pages Router) et les contraintes du cahier des charges.

2 Arborescence du Projet

Le projet suit une structure modulaire permettant de séparer la logique d'état, les services API et les vues. Voici l'état actuel des fichiers créés ou modifiés lors de mes interventions :

```
src/
          pages/
                index.js      (Page d'accueil)
                register.jsx  (Cree pour la Tache 13)
                lobby.jsx     (Cree pour les Taches 18 a 22)
                _app.js       (Configuration globale)
          store/
                user-store.js (Gestion de l'etat utilisateur)
          styles/
                globals.css   (Styles CSS globaux)
          services/         (Logique d'appel API centralisee)
```

3 Détails des Tâches Effectuées

3.1 Tâche 13 : Inscription Utilisateur (register.jsx)

L'objectif était de permettre la création d'un compte via le Web Service `PUT /user`.

- **Fonctionnement :** Un formulaire React récupère l'email, le nom et le mot de passe.
- **Validations :** Avant l'envoi, le code vérifie que l'email se termine par *@univ-tlse2.fr* et que le nom d'utilisateur contient entre 3 et 28 caractères.
- **Appel API :** Utilisation de la méthode `fetch` asynchrone avec un corps JSON. En cas de succès, l'utilisateur est redirigé vers la page de connexion.

3.2 Phase 4 : Système de Matchmaking (lobby.jsx)

Cette phase cruciale gère la mise en relation des joueurs. Elle regroupe les tâches 18 à 22.

3.2.1 Tâches 18 & 19 : Participation et Liste des Joueurs

Le fichier `lobby.jsx` utilise un état local pour suivre si l'utilisateur participe au matchmaking.

- **Participation :** L'appel à `/matchmaking/participate` enregistre le joueur sur le serveur.
- **Polling :** Un système de rafraîchissement automatique (Polling) a été mis en place via un intervalle de 5 secondes. Il interroge `/matchmaking/getAll` pour mettre à jour la liste des adversaires disponibles en temps réel sans recharger la page.

3.2.2 Tâches 20 & 21 : Gestion des Défis

- **Envoi de requête :** Au clic sur le bouton "Défier", le client envoie une requête `GET` incluant l'identifiant de matchmaking de la cible en paramètre d'URL.
- **Acceptation :** Le polling récupère également les demandes reçues. Si l'utilisateur clique sur "Accepter", le service `acceptRequest` est appelé, créant officiellement le match côté serveur.

3.2.3 Tâche 22 : Redirection Automatique

Pour assurer une expérience fluide, une fonction de vérification d'arrière-plan a été ajoutée. Elle interroge régulièrement `/match/getMatch`. Dès qu'un match est détecté (si l'adversaire a accepté notre défi pendant que nous étions dans le lobby), l'application redirige automatiquement l'utilisateur vers l'URL `/match`.

4 Fonctionnement Technique Global

Tous les appels effectués vers le matchmaking et le jeu nécessitent une authentification.

- **Sécurité :** Le jeton (token) récupéré lors de la connexion est stocké dans le *localStorage*.
- **Headers :** Chaque requête AJAX inclut un header spécifique `www-authenticate` contenant ce token, conformément aux spécifications du backend.
- **Nettoyage :** Pour éviter les fuites de mémoire et les appels inutiles, les intervalles de polling sont systématiquement détruits lorsque le composant est démonté (quand l'utilisateur quitte la page).

5 Documentation de l'utilisation de l'IA

Conformément aux règles du projet, voici la documentation liée à l'assistance par l'IA :

- **Outil :** Gemini 3.1 Pro (Web, Paid Tier).
- **Utilisateur :** Hyacinthe.
- **Prompt(s) significatif(s) :** Demandes de structuration des composants `register.jsx` et `lobby.jsx` selon l'arborescence Pages Router et implémentation du polling pour le matchmaking.

— **Contexte :** Support technique pour la gestion des appels API asynchrones et la synchronisation de l'état du lobby.
— **Analyse personnelle :** L'IA a permis d'implémenter rapidement une structure robuste pour le polling et la gestion des erreurs HTTP. J'ai dû modifier les suggestions initiales pour les adapter strictement à l'arborescence spécifique de notre projet et à la méthode d'authentification par header imposée par le sujet.